LECTURE

Using Built-In Functions to Standardize Code

Built-In Functions

Simplify infrastructure code by providing built-in helpers to manipulate strings, numbers, lists, and more.

- Reduces repetitive tasks
- Simplifies logic
- Ensures consistent patterns in deployments

Mastering core functions creates more maintainable, reliable, and readable Terraform configurations.

Core Terraform Functions

Numeric Functions

Quickly evaluate numeric values (e.g., finding the smallest subnet size).

Examples:

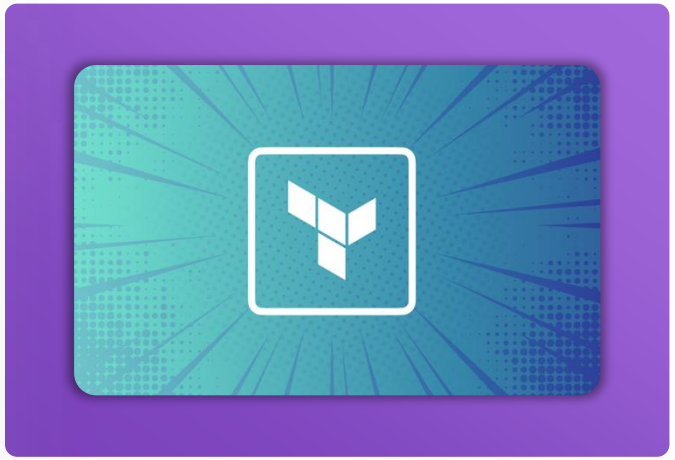
`mix` **&** `max`

String Functions

Concatenate or encode data for resource names, user data, etc.

Examples:

`join` **&** `split`



Type Conversions

Ensure consistent data structures across your configuration and avoid type mismatch issues.

Examples:

`toset` **&** `tolist`

General Syntax for Terraform Built-In Functions

Syntax: `function_name(argument1, argument2, ...)`

- **Example 1:** `upper("hello")` → **returns** `"HELLO"`
- **Example 2:** `min(4,7,2,9,5)` → **returns** `2`
- **Example 3:** `join("-", "hello", "terraform")` → `"hello-terraform"`

Numeric Functions

Determining the largest or smallest value among different variables/outputs

- **Dynamic Resource Allocation:** Adjust CPU, memory, or storage sizes without hardcoding.
- **Automated Calculations:** Remove guesswork by letting Terraform handle numeric logic.

```
# Max: Returns the highest value in a list of numbers
max(10, 4, 7) → 10

# Min: Returns the lowest value in a list of numbers
min(10, 4, 7) → 4

# ####################

variable "number" {
  type    = number
  default = 15
}

max(10, 4, var.number) → 15
```

String Functions

Manipulate and transform textual data — enabling consistent naming, simplified automation, and more maintainable configurations.

```
# Join: Concatenates a list of strings into a single string.
join("-", ["prod", "web", "us-west-1"]) → prod-web-us-west-1

# upper / lower: Purpose: Converts a string to uppercase/lowercase
upper("example") → EXAMPLE
lower("HELLO") → hello

# replace: Substitutes part of a string with another value.
replace("123-abc", "abc", "xyz") → 123-xyz

# base64encode: Encodes string to Base64, often required for user data
base64encode("my-secret-data") → "bXktc2VjcmV0LWRhdGE="
```

Network Functions

Specialized functions to work with IP addresses and subnets

- **Automate CIDR Calculations:** Build subnet blocks and host addresses seamlessly.
- **Reduce Manual Errors:** Eliminate hand-calculating IP ranges.
- **Scalable Network Design:** Easily create multiple subnets in large or complex deployments.

```hcl
# Create a VPC with a /16 CIDR
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Create a public subnet in the first available range
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  availability_zone = "us-east-1a"
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 3, 0)
}

# Create a private subnet in the second available range
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  availability_zone = "us-east-1b"
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 3, 1)
}
```

Type Conversion Functions

Convert collections of data into different types for inputs or outputs

- `toset` / `tolist`
  Convert collections into lists or sets
- `tomap`
  Convert a list or object to a map
- `tostring`
  Convert the argument to a string

```hcl
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1a", "us-east-1b"]
}

locals {
  unique_zones = toset(var.availability_zones)
}

resource "aws_subnet" "example" {
  for_each = local.unique_zones

  vpc_id            = aws_vpc.main.id
  availability_zone = each.value
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 2, each.key)
}
```

LECTURE

Enhancing Code with Dynamic Values

Variables Recap

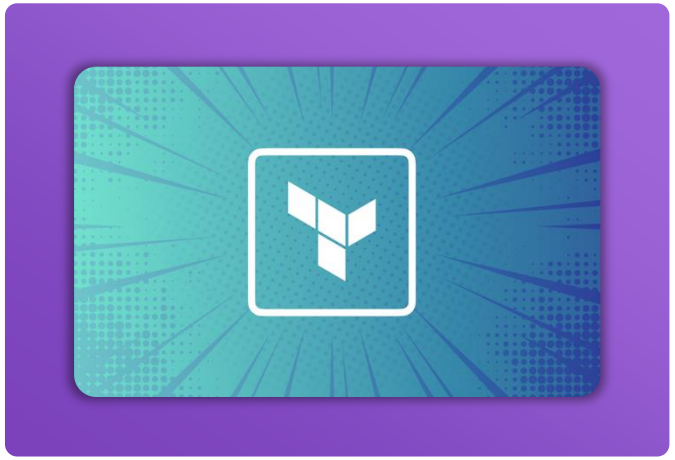
Dynamic Inputs for Terraform Code

Input variables let you customize Terraform configurations without changing the source code or hardcoding values

Variable Types

Variable types include string, number, bool, map, set, list.

Set values via defaults, ENV, .tfvars, and command line



Variables Keep Your Code DRY

Pass in different values to create unique resources

Source: HashiCorp State of Cloud Strategy Survey

Data Sources - Reap

Pull External Information into Terraform

Provider-Specific Info

Data sources (blocks) query provider-specific information about existing resources so we can use it in our code

Common Use Cases

Dynamic lookups, reference existing infrastructure, and avoid hardcoding or repeating information

No Changes

Data sources don't create or make any changes to infrastructure. They only read information and pull it into Terraform

Combining Variables and Data Sources Can Create More Flexible Code

Dynamic code doesn't use static values – it uses interpolation, meta-arguments, functions, local values, and other methods to adapt to changing needs.

Static Inputs & Variables

main.tf

```
resource "aws_instance" "example" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}

resource "aws_vpc" "example" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "hardcoded-vpc"
  }
}
```

main.tf

```
resource "github_repository" "example" {
  name        = "my-static-repo"
  description = "A static name"
  visibility  = "public"
}

resource "github_team" "example" {
  name        = "hardcoded-team"
  description = "A team with a name"
}
```

Dynamic Code

```
resource "aws_instance" "example" {
  ami = data.aws_ami.ubuntu.id
  instance_type = local.instance_type

  tags = {
    Name = "${var.app}-${local.env}-server"
  }
}

resource "aws_vpc" "example" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = "vpc-${local.env}-${data.aws_region}-${var.network}"
  }
}
```

Interpolation

This ${ . . . } sequence is called an interpolation. It evaluates the expression between the markers, converts the result, and inserts it into the string

main.tf

```
variable "name" {
  type    = string
  default = "Bryan"
}


"Hello, ${var.name}!"
Hello, Bryan!


"My name is ${var.name}, I'm your instructor!"
My name is Bryan, I'm your instructor!
```

Interpolation

main.tf

```
variable "environment" {
  type    = string
  default = "dev"
}

data "aws_region" "current"{}
data "aws_caller_identity" "current" {}


name = "server-${data.aws_region.current.name}-${var.environment}"
server-us-east-1-dev

bucket_name = "s3-${data.aws_caller_identity.current.account_id}-backups"
s3-9876543210-backups
```

LECTURE

Using Locals to Avoid Code Duplication

When Code Duplication Strikes

```hcl
resource "some_resource" "server-abc" {
  argument     = var.argument
  agument_type = var.argument_type

  tags = {
    Name        = "${var.app_na
    ManagedBy   = "Terraform"
    Team        = var.team
    Environment = var.environme
    ID          = "server-${loc
  }
}
```

```hcl
resource "some_resource" "server-xyz" {
  argument     = var.argument
  agument_type = var.argument_type

  tags = {
    Name        = "${var.app_name}-${var.server}-xyz"
    ManagedBy   = "Terraform"
    Team        = var.team
    Environment = var.environment
    ID          = "server-${local.env}-${data.aws_region}-${var.server[1]}"
  }
}
```

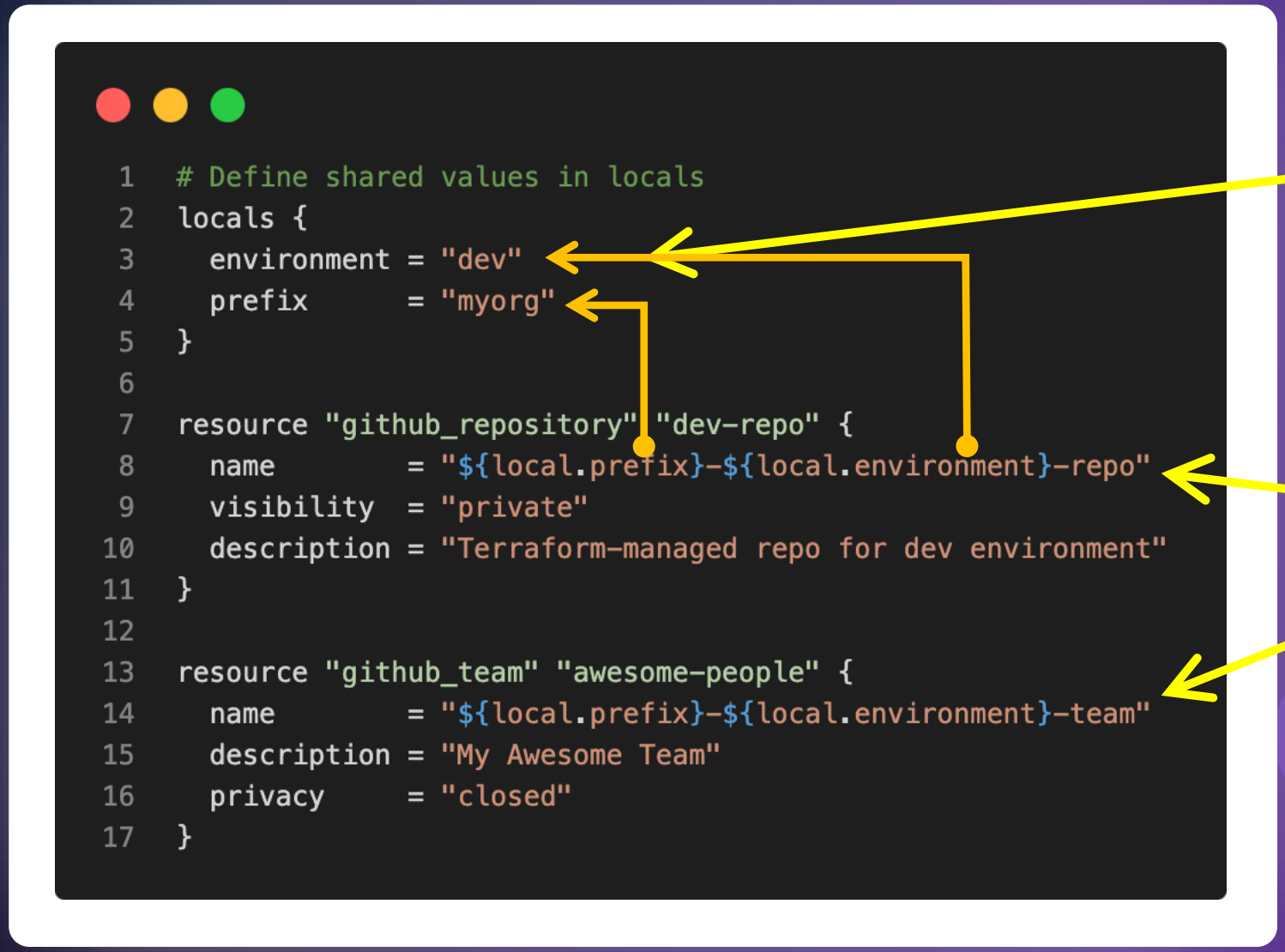
What are Local Values?

Named values that you can reference in other parts of your Terraform code

- Most often referred to as "locals," think of locals like short-term memory for your Terraform code
- They centralize and store values you use often, like names, environments, or shared tags
- Local values can be helpful to avoid repeating the same values or expressions multiple times in a configuration

```
1  locals {
2    app_name    = "my-app"
3    environment = "dev"
4  }
5
```

Basic Syntax - Local Values



```hcl
# Define shared values in locals
locals {
  environment = "dev"
  prefix      = "myorg"
}

resource "github_repository" "dev-repo" {
  name        = "${local.prefix}-${local.environment}-repo"
  visibility  = "private"
  description = "Terraform-managed repo for dev environment"
}

resource "github_team" "awesome-people" {
  name        = "${local.prefix}-${local.environment}-team"
  description = "My Awesome Team"
  privacy     = "closed"
}
```

locals block

Using a locals block

Basic Syntax - Local Values

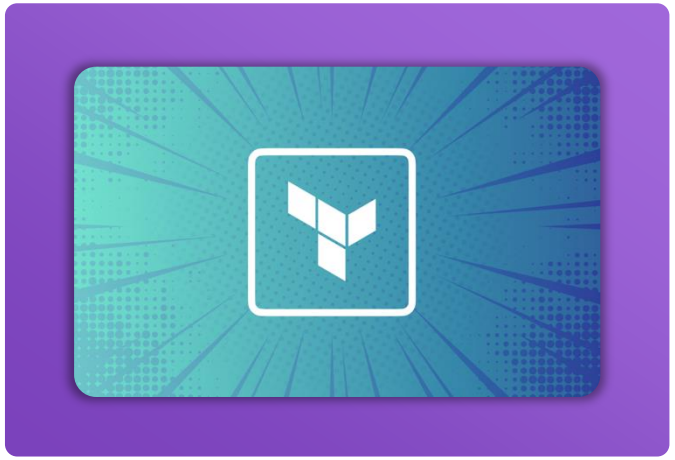
Benefits of Locals

Centralized Logic

The ability to easily change the value in a central place is the key advantage of local values

Clearer Code

Reduces clustter and makes it obvious where a value comes from



Less Risk

Changing a local is less error-prone than updating multiple hardcoded values

When Should I Use Local Values?

- When you need to combine variables or data sources to create dynamic values, locals help keep your code readable and consistent.

- If the same value appears multiple times (like a prefix, environment name, or common tag), define it once in locals instead of repeating it across multiple resources.

- Instead of embedding lengthy calculations or conditions directly in resource definitions, you can compute them in locals first and reference them where needed.

`count` Meta-Argument

Specify a number of instances for a resource

- Allows Terraform to create multiple identical resources using a single block
- Assigns an index (starts at 0) to reference each instance
- Use `count` to create resources that are identical but need to be repeated or when a variable controls the number of resources
- Reference the resource using `<type>.<name>[index]`

```hcl
variable "vm_count" {
  type    = number
  default = 3
}

resource "azurerm_virtual_machine" "example" {
  count               = var.vm_count
  name                = "vm-${count.index}"
  location            = "East US"
  resource_group_name = "my-resource-group"
  vm_size             = "Standard_D2s_v3"

  os_profile {
    computer_name  = "vm-${count.index}"
    admin_username = "adminuser"
  }
}
```

Variable controlling the number of resources

`azurerm_virtual_machine.example[0]`

`for_each` Meta-Argument

A More Flexible Way to Manage Multiple Resources

Creates Multiple Resources

Pulls in values from a map or set rather than a fixed number

Resources Get a Unique Key

Rather than an index, each resource gets a unique key as a reference, making it more stable than count

Ideal for Managing Unique Configurations

Use it when resources need different names, tags, or settings

```
1  variable "vms" {
2    type = map(string)
3    default = {
4      "vm1" = "Standard_D2s_v3"
5      "vm2" = "Standard_D4s_v3"
6      "vm3" = "Standard_B2ms"
7    }
8  }
9
10 resource "azurerm_virtual_machine" "example" {
11   for_each            = var.vms
12   name                = each.key
13   location            = "East US"
14   resource_group_name = "my-resource-group"
15   vm_size             = each.value
16
17   os_profile {
18     computer_name  = each.key
19     admin_username = "adminuser"
20   }
21 }
azurerm_virtual_machine.example["vm1"]
```

Variable controlling the resources

Key

Value